Evento Claude for AI Engineering · Universidad Iberoamericana
Duración 55 min de contenido + 5 min de colchón
Ponente Dante F. Bazaldua Huerta — CTO de Spot2, exalumno de UIA

MCP: conceptos, primitives y diseño de tools

Un LLM es una función de texto a texto. Todo lo demás — saber, actuar, verificar — hay que dárselo. La charla recorre cómo se le da, hasta llegar a MCP, y luego cómo diseñarlo bien.

0. Apertura — 1 min

Escribir un MCP server es fácil. Que el modelo elija la tool correcta, con los parámetros correctos, no lo es. Empezamos por una pregunta anterior: por qué un modelo necesita herramientas.

1. Por qué un LLM necesita herramientas — 8 min

Un LLM recibe texto y produce texto. Eso es todo. De ahí salen cinco límites: conocimiento congelado, sin acceso a lo privado, sin efectos en el mundo, cómputo no determinista, no puede verificar.

Esas capacidades se han ido resolviendo por acumulación, no por reemplazo: meter todo en el prompt → RAG (resuelve *saber*) → function calling (resuelve *hacer*, pero definido en build time) → acceso a shell/VM (máxima generalidad, sin estructura ni permisos finos) → tools dinámicas, descubiertas en runtime.

Con tools dinámicas, las capacidades se componen en runtime: quien las aporta es el dueño del sistema, no el dueño del agente. Eso es lo que **MCP** estandariza — el problema de M clientes × N sistemas se vuelve M+N. Es a los agentes lo que HTTP es a las aplicaciones: un contrato, no una librería.

Lo que no resuelve: un prompt malo, datos sucios, autorización, ni que el modelo entienda tu dominio. Y crea un problema nuevo: elegir bien entre 60 tools de seis servers distintos no es trivial.

2. CLI, API REST, function calling y MCP — 6 min

Dimensión	CLI	Function calling	MCP
Quién invoca	El humano / modelo generando texto	El modelo	Modelo (Tools), app (Resources), usuario (Prompts)
Descubrimiento	El usuario lee la doc	Hardcodeado en la app	En runtime, vía `tools/list`
Quién escribe la integración	El usuario	El dueño del agente	**El dueño del sistema**

MCP vs function calling: no son competidores, son capas distintas. Function calling es el mecanismo (cómo un modelo emite una llamada); MCP es el protocolo (cómo esas capacidades se publican y consumen entre procesos).

"**Esto es REST otra vez":** parcialmente cierto a nivel transporte, falso a nivel diseño — el contrato está escrito para un lector que infiere, no uno que lee documentación. Regla práctica: si tu server se ve como un `swagger.json` traducido, tienes una API con otro nombre.

3. Las tres primitives y el modelo de control — 8 min

Tools, Resources y Prompts se declaran igual y se listan por el mismo protocolo. Lo que los separa: **quién decide cuándo ocurre la acción.**

Primitive	Controlado por	Cuándo elegirlo
Tools	El modelo	Todo lo que tiene efecto: escribir, borrar, enviar
Resources	La aplicación	Contexto que el host decide inyectar: schemas, docs
Prompts	El usuario	Workflows empaquetados, vía slash commands

Regla práctica. Si el modelo decide, es una Tool. Si la aplicación decide, es un Resource. Si el usuario decide, es un Prompt. La mayoría expone todo como Tools porque es el camino de menor resistencia — y ahí se pierde control de compliance (Resources) y workflows empaquetados (Prompts, la de mejor relación esfuerzo/impacto).

4. Cómo se escribe un server — 8 min, hands-on

Una demo en vivo con Python + FastMCP: un **tool** con type hints + docstring (el SDK genera el schema; el docstring es lo único que el modelo lee), un **resource** con su URI, un **prompt**. Probado en el MCP Inspector y conectado a Claude Code, observando la selección de tools.

5. Diseño de tools — 13 min

El modelo no lee tu código ni tu README. Lee **nombre + descripción + schema**. Todo el diseño ocurre en esos tres campos.

- **Agrupa por intención, no por endpoints.** 5-15 tools por server. `query_db(sql)` traslada el trabajo de diseño al modelo. Tools que se solapan compiten entre sí.
- **Schemas estrictos y side-effects explícitos.** Enums, límites, `additionalProperties: false`. Di en la descripción qué es reversible: "esto borra permanentemente", "esto envía correo a un cliente real".
- **El error es una instrucción.** `{"error": "invalid input"}` hace reintentar en ciclo. `{"error": "fecha debe ser YYYY-MM-DD, recibí '15 de marzo'"}` se corrige en un turno.
- **El contexto es presupuesto.** 40 tools son tokens en cada llamada, en cada conversación.
- **Composición entre servers.** Nombres genéricos (`search`, `create`, `list`) colisionan a nivel ecosistema. Usa un prefijo de dominio.

Cómo validar. Escribe 10 prompts realistas → córrelos contra tu server → cuenta aciertos de tool + parámetros → itera **descripción y schema**, no el system prompt → repite hasta que deje de fallar.

6. Dónde vive el estado — 4 min

El protocolo es stateless: cada request es autosuficiente. El estado no desapareció, se volvió explícito — emites un *handle* desde una tool y el modelo lo pasa de vuelta como argumento. Eso lo hace visible para el actor que tiene que razonar sobre él.

Regla práctica. Diseña cada tool para que se ejecute de forma independiente. Si hay continuidad, hazla visible como parámetro.

7. Seguridad al conectar sistemas internos — 5 min

- **Least privilege por tool**, no por server. Un server con credencial de admin y 12 tools tiene 12 caminos de escalación.
- **Valida siempre los inputs**, aunque el schema ya los declare.
- **Prompt injection vía resultados de tools:** lo que devuelve tu sistema interno entra al contexto del modelo.
- **Tool poisoning:** la descripción que aprobaste puede no ser la que corre después.
- **Confirmación del usuario** para acciones destructivas o costosas, y **sandboxing** cuando el server toca datos reales.

8. Cierre — 2 min

1. Escribe un server con las tres primitives, no solo tools.
2. Pruébalo con el Inspector.
3. Conéctalo a Claude Code / Claude Desktop.
4. Corre la evaluación de 10 prompts.
5. Itera descripciones y schemas hasta que deje de fallar.

Presupuesto de tiempo

Bloque	Min
0. Apertura	1
1. Por qué un LLM necesita herramientas	8
2. Diferencias de ejecución	6
3. Las tres primitives y el modelo de control	8
4. Cómo se escribe un server	8
5. Diseño de tools	13
6. Dónde vive el estado	4
7. Seguridad	5
8. Cierre	2
Total contenido	**55**
Colchón	5